

MIMIR: Verifiable, Privacy-Preserving Exam Generation with Local Language Models

[Author 1], [Author 2], [Author 3]

[Affiliation to be completed]

[City], Hungary

[email addresses]

Abstract—Teachers increasingly use large language models (LLMs) to draft exam questions, but the typical “upload a document, get a quiz” workflow gives no guarantee that the keyed answer is supported by the source, sends the material to third-party services, and is rarely evaluated in languages other than English. We present MIMIR, an open microservice system that generates multiple-choice exams from a teacher’s own documents in Hungarian and English. MIMIR replaces single-shot retrieval-augmented generation with a *blueprint* pipeline: a planner allocates concepts, question types and Bloom levels to slots; each question is written from its own retrieved passages with chunk citations; and a verifier chain checks grounding, distractors and blind answerability, feeding failures back to the generator. The system runs on an 8 GB consumer GPU, keeps uploaded content only for the duration of a job, and makes human review mandatory before export. We describe a reproducible evaluation protocol with 12 experiment arms, a 39-document bilingual dataset with 326 human-written reference questions, an LLM judge from a different model family, and a blind teacher rating. Building the harness already exposed two defects that silently invalidated earlier measurements: a hard-coded fallback exam that was counted as model output, and a chunker encoder that truncated every document after 512 tokens—an estimated 68–71 % of the average dataset document. The full comparison of the arms is reported with the protocol described here.

Index Terms—automatic question generation, retrieval-augmented generation, LLM-as-a-judge, educational technology, Hungarian NLP, GDPR, EU AI Act

I. INTRODUCTION

Writing good multiple-choice questions is slow: the stem must be unambiguous, exactly one option must be correct according to the course material, and the distractors must be plausible but provably wrong. Automatic question generation (AQG) has a long history in education [1], and general-purpose LLMs have made it accessible to every teacher. The common implementation is naive retrieval-augmented generation (RAG) [2]: split the document, retrieve the top- k passages for the teacher’s request, and ask the model for the whole exam in one call.

This design has three weaknesses in a school setting. *Correctness is not checked*: nothing verifies that the keyed option is supported by the source or that distractors are actually false. *Control is weak*: the requested number of questions, types and difficulty are only suggested in the prompt. *Privacy is an afterthought*: teaching materials may contain personal data, yet they are sent to external APIs and often kept. Evaluation is a

further gap: published AQG results are dominated by English, while our users teach in Hungarian.

MIMIR addresses these points with the following contributions:

- A **blueprint pipeline** (plan → retrieve per slot → generate with citations → verify → assemble) that guarantees the requested exam shape by construction and verifies every question before a human sees it (Section IV).
- A **privacy-by-design deployment** on an 8 GB GPU: zero retention of uploaded content, an optional local-only mode, a visible processing location and model choice before each upload, and mandatory human review (Section V).
- A **reproducible bilingual evaluation protocol**: 12 arms, 39 documents with 326 reference questions, a cross-family LLM judge, document-level statistics and a blind teacher rating (Section VI).
- **Diagnostic findings** from building the harness: two defects that made earlier end-to-end measurements invalid, and how the harness now detects them (Section VII).

II. RELATED WORK

Surveys of educational AQG [1] note that most systems target English and are evaluated with reference-overlap metrics or small human studies that rarely check answer correctness. RAG [2] grounds generation in retrieved text; graph-based variants build an entity graph over the corpus to answer questions that need global context [3]. Hybrid retrieval combines lexical BM25 [4] with dense embeddings, typically fused with reciprocal rank fusion (RRF) [5]. For Hungarian, multilingual E5 embeddings [6] are a strong open choice, and MILQA [7] provides question–answer pairs over Hungarian Wikipedia; EduQG [8] provides multiple-choice questions over English OpenStax textbooks. Using LLMs as judges scales evaluation and correlates reasonably with humans [9], but judges prefer their own generations [10], which motivates judging with a model from a different family and validating the judge against teachers. Cognitive levels are commonly expressed with the revised Bloom taxonomy [11].

III. SYSTEM ARCHITECTURE

MIMIR is a set of FastAPI microservices behind an Nginx gateway, with a React/TypeScript frontend (Fig. 1). *Well-spring* extracts text from PDF, DOCX and plain-text files.

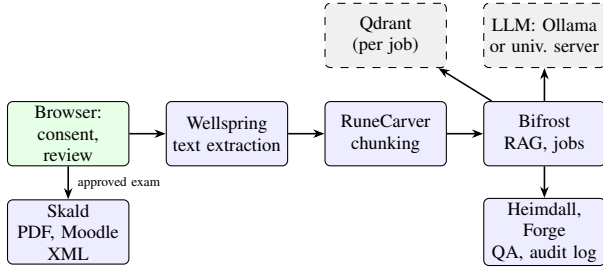


Fig. 1. MIMIR services behind the Nginx gateway. The draft exam lives in the browser during review; uploaded content lives only in service memory and a per-job vector collection that is cleared when the job ends.

RuneCarver splits it into semantic chunks: sentences are embedded with multilingual-E5 [6], and cuts are placed where the cosine distance between neighbouring sentences exceeds a percentile (or mean + $k\sigma$) threshold, within a target chunk size. *Bifrost* embeds and indexes the chunks in Qdrant, retrieves context and runs the generation job, streaming from the university GenAI server or a local Ollama model. *Heimdall* validates the output schema and scores quality. *The Forge* orchestrates background work and keeps a metadata-only audit log. *Skald* renders the reviewed exam as PDF or Moodle XML.

Hardware target. The whole stack, including a 7B generator (Qwen2.5-7B [12] in Ollama) and the E5 encoder, is sized for an 8 GB consumer GPU; the embedding models run on CPU in float32. Generation jobs are asynchronous: POST /generate returns a job id, and GET /status reports the stage (retrieving, generating, validating), the progress derived from the length of the streamed output, and a remaining-time estimate blended from the observed pace and the median duration of recent jobs, kept separately for local and server models. The frontend turns this into a step list, a progress bar and a countdown, so a teacher waiting one or two minutes knows what is happening.

IV. BLUEPRINT PIPELINE

The naive baseline (arm E0) retrieves the top three chunks for the request and asks for the whole exam in one call. The blueprint pipeline (Fig. 2, Algorithm 1) instead makes the exam structure explicit.

Planner. One LLM call proposes $n + 3$ concepts that cover the document, each with the chunk identifiers that support it and an importance score. Slots are filled greedily, preferring concepts that touch chunks not yet covered, so the exam spreads over the whole document; the three spare concepts serve as replacements. Each slot receives a question type and a Bloom level [11] drawn by largest-remainder allocation from a difficulty-dependent mix (easy: remember/understand; medium: understand/apply/analyse; hard: apply/analyse/evaluate). The requested count and types are therefore met by construction, not by prompting.

Per-slot retrieval and generation. For each slot the concept’s own chunks are combined with the top- k search hits for the concept (dense E5 search, or BM25 and dense fused with

TABLE I
VERIFIER CHECKS, IN EXECUTION ORDER. CHEAP RULE CHECKS RUN FIRST SO THAT MALFORMED QUESTIONS NEVER COST AN LLM CALL.

Check	LLM	Feedback to the generator on failure
Shape	–	MCQ: 4 options, 1 key; TF: 2 options; open: 1 key
Meta-reference	–	Rephrase as a standalone exam question
Citations	–	Cite at least one chunk from the given context
Grounding and distractors	1	Key not supported by the cited chunks, or a distractor may be true; question ambiguous
Blind answer	1	An independent solver chose another option; make the key unambiguous
Duplicate	–	Too similar to an accepted question

Algorithm 1 Blueprint generation for one exam

Require: chunks C , count n , types T , difficulty d , retries r

- 1: $K \leftarrow \text{PLAN}(C, n + 3)$ $\triangleright$ concepts with chunk ids
- 2: $S \leftarrow \text{ALLOCATE}(K, n, T, \text{BLOOMMIX}(d))$
- 3: **for all** slot $s \in S$ **do**
- 4: **for** $a = 0 \dots r$ **do**
- 5: $X \leftarrow \text{RETRIEVE}(s.\text{concept}) \cup s.\text{chunks}$
- 6: $q \leftarrow \text{GENERATE}(s, X, \text{feedback})$
- 7: $\text{feedback} \leftarrow \text{VERIFY}(q, X)$ $\triangleright$ shape, cites, grounding,
- blind
- 8: **if** $\text{feedback} = \emptyset \wedge \neg \text{DUP}(q)$ **then** accept q ; **break**
- 9: **end if**
- 10: **end for**
- 11: **if** not accepted **then** try a spare concept, else keep best q as unverified
- 12: **end if**
- 13: **end for**
- 14: **return** questions ordered easy $\rightarrow$ hard, with citations

RRF [5] in the hybrid arm). The generator writes *one* question per call and must cite the chunk identifiers that support the key.

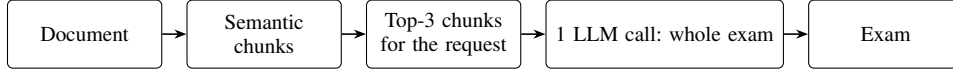
Verifier chain. Checks run cheapest first: shape (four options with one key for MCQ), meta-references (“according to the text...”), valid citations, then an LLM check that the cited chunks support the key and refute every distractor, and finally a *blind answer test*: the model answers the question from the cited text without seeing the key and must pick the keyed option. A failed check is returned to the generator as feedback. After the retry budget the slot switches to a spare concept; as a last resort the best attempt is kept and flagged `verified=false`, so the teacher sees which questions need attention. Near-duplicates (embedding similarity > 0.9) are regenerated. Table I lists the checks and the feedback each one returns.

Concept graph (E3). A GraphRAG-style [3] variant extracts concepts, definitions and typed relations per 3,000-character batch. Graph concepts replace the planner call, sibling concepts (same parent, different fact) are offered as distractor ideas, and 1-hop neighbour chunks join the context.

V. PRIVACY AND TRUST BY DESIGN

MIMIR is built for use under the GDPR [13] and the transparency and human-oversight duties of the EU AI Act [14].

E0 – naive RAG



E1–E3 – blueprint

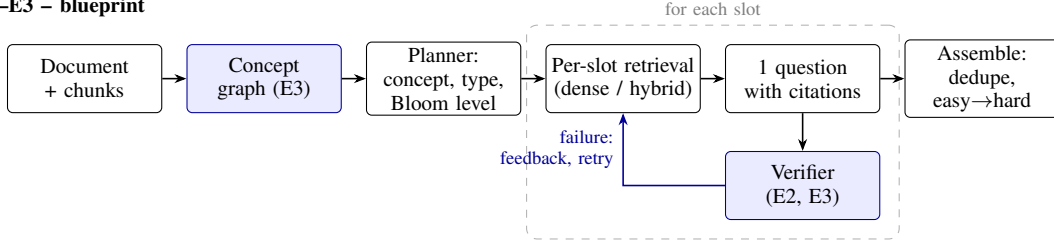


Fig. 2. Naive RAG (top) versus the blueprint pipeline (bottom). Blue: components switched on and off by the experiment arms. The blueprint fixes the exam structure before any question is written and verifies each question against its own cited context.

Zero retention. Uploaded bytes, extracted text, chunks, embeddings and prompts exist only in service memory for the duration of a job; Bifrost clears the vector store when a job ends or fails, and job results expire after 60 minutes. The audit log stores the job id, model, prompt version, quality score and SHA-256 fingerprints and lengths of the request and context—never content. Access logs keep shortened IP addresses and no query strings. Nothing from uploads is backed up.

Processing location. A `LOCAL_ONLY` switch disables every external model call. Otherwise the upload screen names where the document will be processed, a model selector groups models into “university server” and “local”, and server models carry a “data leaves this server” badge. The upload requires an explicit confirmation that the user may process the document and that it contains no special-category or student data.

Human oversight and transparency. Generated questions open in an editor (reorder, edit, change the key, regenerate one question, view the cited source passage) and cannot be exported without approval. The exported PDF states that the content was produced with an AI assistant and names the model. Saving a test is opt-in, with a 12-month maximum retention. A compliance pack (record of processing, retention schedule, DPIA screening, breach procedure) with a draft data-processing-agreement template and a legitimate-interest assessment is maintained with the code.

VI. EVALUATION PROTOCOL

The harness (`tests/eval`) runs in three steps: *run* generates exams for every document $\times$ seed of an arm, *score* computes metrics and calls the judge, and *analyze / report* produce tables, tests and figures. Each run stores its resolved configuration, git commit and dataset version; judge answers are cached, so re-scoring is free.

Dataset. Table II summarises the 39 documents. EduQG [8] and MILQA [7] importers pick short, medium and long documents and keep only reference questions whose evidence sentences occur in the document; reference questions are used for gold similarity and retrieval recall, not as targets. The dataset is frozen as `eval-v1` before the full run.

TABLE II
EVALUATION DATASET (39 DOCUMENTS, 326 REFERENCE QUESTIONS).

Source	Lang	Docs	S/M/L	Median chars	Ref. Q
Own teaching texts	HU	3	3/0/0	1,923	12
OpenStax via EduQG	EN	12	4/4/4	16,752	74
Wikipedia via MILQA	HU	24	8/8/8	8,072	240
Total		39	15/12/12	7,919	326

TABLE III
EXPERIMENT ARMS. EACH CHANGES ONE FACTOR.

Arm	Configuration
E0	Naive RAG: top-3 chunks, whole exam in one call (reference)
E1	Blueprint: planner + per-slot retrieval, one question per call
E2	E1 + verifier chain, 1 retry, blind answer test (“thorough”)
E2f	E2 without the blind answer test (“fast”)
E2h	E2 with hybrid BM25 + dense retrieval (RRF)
E3	E2 + concept graph
E4 / E4b	E0 / E2 with a large model on the university server
E5a	E0 with the legacy 512-token chunk encoder
E5b / E5c	E0 with a mean+1.5 σ cut rule / fixed 800-char chunks

Arms. Table III lists the experiment arms. All local arms use Qwen2.5-7B with temperature 0.2 and a context window of 8,192 tokens. The naive and chunking arms run three seeds per document; the more expensive blueprint arms run one seed; the server arms run on a fixed paired subset of 12 documents (6 HU, 6 EN).

Metrics. *Format compliance* is the share of exams with exactly the requested number and shape of questions; an unparseable output or Bifrost’s hard-coded error exam counts as a failed exam, while infrastructure errors are excluded and retried. The judge scores, per question, *grounding* (the source supports the key, with a quoted sentence), *distractor validity* (each distractor is false and on topic) and four 1–5 rubric scores (correctness, clarity, distractor quality, Bloom fit); in a second call it answers the question *blind* from the source,

giving *blind answerability*. Automatic metrics add the meta-reference rate, duplicate rate, section coverage, similarity to the reference questions, retrieval recall@ k and MRR against the gold evidence sentences, and cost (time, LLM calls, peak VRAM). The *quality index* is the mean of the four measures every arm has: format compliance, grounding, blind answerability and distractor validity.

Judge. The judge is gpt-oss-120b [15], a different model family from the Qwen generators, to avoid self-preference [10]; the harness warns when judge and generator share a family. It sees the chunks each question was written from (or the retrieved context for E0) and runs with low reasoning effort to keep a full run tractable.

Statistics. Documents are the unit of analysis: metrics are averaged over seeds and questions per document. We report means with 10,000-resample bootstrap 95% CIs [16], paired Wilcoxon signed-rank tests [17] over documents with Holm correction [18] for the key steps (E1–E0, E2–E1, E3–E2, E4–E0), and the matched-pairs rank-biserial correlation [19] as effect size. Fewer than six paired documents can never reach $p < 0.05$ with this test, so pilot runs are labelled as such by the report generator.

Teacher rating. Blind rating sheets sample 15 questions per arm round-robin over documents, shuffled per rater, without arm labels, using the same rubric as the judge plus a “would use” item. Agreement between teachers is measured with Krippendorff’s α [20]; judge–teacher agreement with Spearman’s ρ per criterion and Cohen’s κ for judge “grounded” versus teacher correctness ≥ 4 .

VII. RESULTS

A. Diagnostic findings

Building the harness produced two findings that change how earlier measurements must be read.

Fallback exams counted as results. When every model call fails, Bifrost returns a hard-coded one-question exam asking the user to retry. An end-to-end test run on 3 May 2026 reported the generation step for all three pilot documents as successful in 4.0–9.0s, but each “exam” was this fallback (one question instead of the requested four). Because the payload is valid JSON of the right schema, latency and schema checks alone could not reveal it. The fallback is now marked `is_fallback`, the test client flags it, and the harness counts it as a failed exam.

Chunker truncation. RuneCarver embedded the whole document in one encoder pass with a 512-token limit; every sentence after roughly the first page received a zero vector, so cut points beyond that were driven by the size limit, not topic changes. At 4.0–4.5 characters per token the window covers about 2,000–2,300 characters, which means that on average an estimated 68–71% of each dataset document’s text (median 71–74%) was chunked without semantic signal, and 31–33 of the 39 documents lost more than half. The encoder now works in ≤ 512 -token windows (identical output on short texts; covered by a unit test); arm E5a keeps the old encoder to

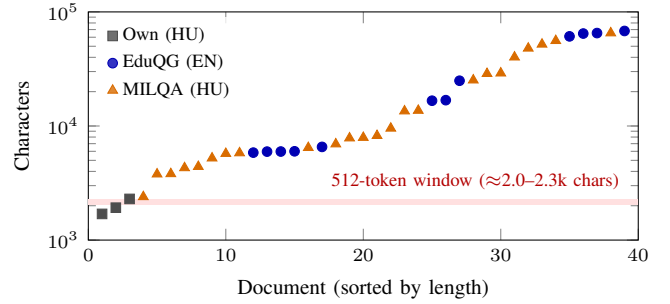


Fig. 3. Length of the 39 dataset documents. With the legacy encoder, only text inside the shaded band received semantic embeddings; 35 documents are longer than 3,000 characters.

TABLE IV
STAGE LATENCY OF ONE NAIVE END-TO-END RUN (27 MARCH 2026).

Service	Stage	Time (s)
Wellspring	Text extraction (1,221 chars)	0.039
RuneCarver	Semantic chunking (3 chunks)	1.303
Bifrost	Embedding + indexing	0.559
Bifrost	LLM generation (1 question)	13.478
Skald	PDF rendering	0.009
Total		15.387

measure the effect. Fig. 3 shows how far the dataset documents extend past the old encoder window.

B. Naive pipeline latency

Table IV shows a successful end-to-end run of the naive pipeline on a 1,221-character Hungarian text (3 chunks, 1 question, university GenAI server). Model generation dominates: 88% of the 15.4s total. This motivates the asynchronous job design and the progress and time-estimate display, and it bounds the cost of the blueprint pipeline, which makes roughly 3–5 LLM calls per question instead of one call per exam.

C. Comparison of the arms

Table V is the main comparison. It is produced by the `report` command from the frozen `eval-v1` run; the values shown in red are placeholders until that run and the teacher rating are complete. We commit to reporting it in this form, including null results.

The analysis answers four questions: whether explicit planning improves format compliance and coverage over naive RAG (E1 vs. E0); whether verification improves grounding and blind answerability, and at what cost in time and calls (E2 vs. E1, E2f); whether hybrid retrieval and the concept graph add to that (E2h, E3 vs. E2); and whether a 7B local model with the blueprint pipeline can match a large server model with the naive one (E2 vs. E4), which decides whether a privacy-preserving local deployment is viable. Results are also broken down by language, since the Hungarian and English documents differ in source and length and the differences are therefore descriptive.

TABLE V
QUALITY BY ARM (DOCUMENT MEANS, 95 % BOOTSTRAP CI FOR THE INDEX). RED: PENDING FULL RUN.

Arm	Format	Ground.	Blind	Distr.	Index [CI]	min/exam
E0	-	-	-	-	-	-
E1	-	-	-	-	-	-
E2	-	-	-	-	-	-
E2f	-	-	-	-	-	-
E2h	-	-	-	-	-	-
E3	-	-	-	-	-	-
E4	-	-	-	-	-	-
E4b	-	-	-	-	-	-
E5a	-	-	-	-	-	-

D. Implementation and quality assurance

The evaluation harness has 32 unit and end-to-end tests; the end-to-end tests run the full run→score→rate→analyze→report flow with fake services and a mock LLM, so the statistics and report tables are tested without a GPU. The job tracker behind the progress display and the model endpoints are covered by service tests with a fake clock. The frontend is typed against an OpenAPI contract and has 48 component and contract tests and 17 browser end-to-end tests, including an automated accessibility scan (axe) in each of the 12 colour-palette and light/dark combinations; a CI script checks WCAG AA contrast for all of them, and a key-parity check keeps the Hungarian and English interface strings (669 keys) in sync. These checks keep the system usable while the evaluation changes the pipeline underneath it.

VIII. DISCUSSION AND LIMITATIONS

Measurement before modelling. The two diagnostic findings are the most transferable lesson. Both defects produced plausible artefacts—a fast, schema-valid response and a normal-looking chunking—and would have biased any comparison built on them. Explicit failure states (a flagged fallback, per-stage status), before/after arms for every fix and a harness with end-to-end tests with fake services and a mock LLM made them visible.

Judge validity. An LLM judge is a proxy. We mitigate self-preference by using another model family, anchor grounding with a quoted evidence sentence, and validate the judge against blind teacher ratings; if agreement is low, the teacher ratings take precedence. Showing the judge only the question’s own context makes it faster but can miss contradictions elsewhere in the document.

Scope. The questions are multiple choice; true/false and open questions are supported by the system but not evaluated here. Documents are the unit of analysis, so 39 documents limit the power for small effects, and the server arms use a 12-document subset. Wikipedia and OpenStax text is cleaner than typical lecture notes. The Hungarian and English subsets come from different sources, so language effects are confounded with source. Finally, real authentication and per-user owner-

ship checks are still in development; until then the service is operated for a closed user group.

IX. CONCLUSION

MIMIR shows that exam generation from teachers’ own material can be structured, verifiable and privacy-preserving on commodity hardware: a blueprint guarantees the requested exam, a verifier chain checks every question against its cited source, uploaded content is not retained, and a teacher approves every question before use. Its evaluation protocol—bilingual data with reference questions, single-factor arms, a cross-family judge validated by teachers, and document-level statistics—already caught two defects that would have invalidated a naive comparison. The code, configurations and dataset manifests are released with the system so that the comparison can be reproduced and extended.

ACKNOWLEDGMENT

[To be completed: funding, access to the university GenAI service, teacher raters.]

REFERENCES

- [1] G. Kurdi, J. Leo, B. Parsia, U. Sattler, and S. Al-Emari, “A systematic review of automatic question generation for educational purposes,” *International Journal of Artificial Intelligence in Education*, vol. 30, no. 1, pp. 121–204, 2020.
- [2] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, “Retrieval-augmented generation for knowledge-intensive NLP tasks,” in *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 9459–9474.
- [3] D. Edge, H. Trinh, N. Cheng, J. Bradley, A. Chao, A. Mody, S. Truitt, and J. Larson, “From local to global: A graph RAG approach to query-focused summarization,” *arXiv preprint arXiv:2404.16130*, 2024.
- [4] S. Robertson and H. Zaragoza, “The probabilistic relevance framework: BM25 and beyond,” *Foundations and Trends in Information Retrieval*, vol. 3, no. 4, pp. 333–389, 2009.
- [5] G. V. Cormack, C. L. A. Clarke, and S. Buettcher, “Reciprocal rank fusion outperforms Condorcet and individual rank learning methods,” in *Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval*, 2009, pp. 758–759.
- [6] L. Wang, N. Yang, X. Huang, L. Yang, R. Majumder, and F. Wei, “Multilingual E5 text embeddings: A technical report,” *arXiv preprint arXiv:2402.05672*, 2024.
- [7] A. Novák, B. Novák, D. Zombori, G. Szabó, Z. Szántó, and R. Farkas, “A question answering benchmark database for Hungarian,” in *Proceedings of the 17th Linguistic Annotation Workshop (LAW-XVII)*, 2023, pp. 188–198.
- [8] A. Hadifar, S. K. Bitew, J. Deleu, C. Devellder, and T. Demeester, “EduQG: A multi-format multiple-choice dataset for the educational domain,” *IEEE Access*, vol. 11, pp. 20 885–20 896, 2023.
- [9] L. Zheng, W.-L. Chiang, Y. Sheng, S. Zhuang, Z. Wu, Y. Zhuang, Z. Lin, Z. Li, D. Li, E. P. Xing, H. Zhang, J. E. Gonzalez, and I. Stoica, “Judging LLM-as-a-judge with MT-Bench and Chatbot Arena,” in *Advances in Neural Information Processing Systems (Datasets and Benchmarks Track)*, vol. 36, 2023.
- [10] A. Panickssery, S. R. Bowman, and S. Feng, “LLM evaluators recognize and favor their own generations,” in *Advances in Neural Information Processing Systems*, vol. 37, 2024.
- [11] L. W. Anderson and D. R. Krathwohl, Eds., *A Taxonomy for Learning, Teaching, and Assessing: A Revision of Bloom’s Taxonomy of Educational Objectives*. New York: Longman, 2001.
- [12] Qwen Team, “Qwen2.5 technical report,” *arXiv preprint arXiv:2412.15115*, 2024.
- [13] European Parliament and Council, “Regulation (EU) 2016/679 (General Data Protection Regulation),” *Official Journal of the European Union*, L 119, 2016.

- [14] —, “Regulation (EU) 2024/1689 laying down harmonised rules on artificial intelligence (Artificial Intelligence Act),” Official Journal of the European Union, L series, 2024.
- [15] OpenAI, “gpt-oss-120b & gpt-oss-20b model card,” *arXiv preprint arXiv:2508.10925*, 2025.
- [16] B. Efron and R. J. Tibshirani, *An Introduction to the Bootstrap*. New York: Chapman & Hall, 1993.
- [17] F. Wilcoxon, “Individual comparisons by ranking methods,” *Biometrics Bulletin*, vol. 1, no. 6, pp. 80–83, 1945.
- [18] S. Holm, “A simple sequentially rejective multiple test procedure,” *Scandinavian Journal of Statistics*, vol. 6, no. 2, pp. 65–70, 1979.
- [19] D. S. Kerby, “The simple difference formula: An approach to teaching nonparametric correlation,” *Comprehensive Psychology*, vol. 3, 2014.
- [20] K. Krippendorff, *Content Analysis: An Introduction to Its Methodology*, 2nd ed. Thousand Oaks, CA: Sage, 2004.